

Research Assignment: Investigating the Role, Impact, and Relevance of UML in Software Engineering in Modern Software Engineering

Name : Vorng Viseth

Course : Software Engineer

Group : G4

1. Why does UML exist?

1. Why was UML created in Software Engineering?

UML (Unified Modelling Language) was created to solve a major problem in software development: lack of a common way to design and communicate system structure and behaviour.

i. What problems was it trying to solve?

- Poor communication between developers and stakeholders
- Unclear system requirements and design
- Lack of standard documentation
- Difficulty maintaining and scaling systems
- Inconsistent design practices across teams

ii. Were developers struggling before UML?

Yes, Developers often relied only on:

- Textual requirements (which were ambiguous)
- Ad-hoc diagrams (drawn differently by each team)
- Personal interpretation of system design

This led to misunderstandings, bugs, and expensive redesigns.

2. Could software engineering function effectively without UML?

Yes, but with limitations.

Software engineering *can function without UML*, especially in small teams or simple systems. However:

- Communication becomes less structured
- Design quality depends heavily on individual skill
- Documentation is inconsistent or missing

In large systems (like banking or hospital systems), not using UML increases the risk of:

- Miscommunication
- Architecture flaws

- Maintenance difficulty

So, UML is not strictly required, but it acts as a standard “thinking language” for system design.

3. Why do engineers model systems before implementation?

Because modelling helps engineers understand the problem before writing code.

Key reasons:

- Clarifies system structure and behaviour
- Helps detect design issues early
- Improves team alignment
- Reduces wasted development effort

I. Is coding immediately always a good approach?

No.

Jumping directly into coding often leads to:

- Poor architecture
- Rewriting large parts of the system later
- Hidden requirement misunderstandings
- Technical debt accumulation

Modelling first is like drawing a blueprint before building a house. Without it, developers may build something that works—but not something scalable, maintainable, or correct.

2. Impact of UML

1. How does UML improve communication among stakeholders?

UML provides a shared visual language:

- **Developers** - understand system structure and logic
- **Managers** - see system scope and progress
- **Clients** - understand features without technical knowledge
- **Testers** - derive test cases from diagrams

Instead of long technical documents, UML diagrams make system ideas visual and easier to understand across roles.

2. How does UML reduce software development risks?

UML reduces risk by exposing problems early in design:

- **Misunderstood requirements**
Use case diagrams clarify user interactions
- **Wrong architecture**
Class and component diagrams show structure before coding

- **Duplicated modules**
Class diagrams help identify repeated responsibilities
- **Late project delivery**
Better planning and estimation using system models
- **Costly redesign**
Design flaws are detected before implementation, not after deployment

3. How does UML improve maintainability of software?

If a new developer joins after 2 years:

UML helps them:

- Understand system structure quickly
- See relationships between components
- Identify dependencies without reading all code
- Reduce time needed to onboard

Without UML, they must rely only on code, which can be:

- Complex
- Poorly documented
- Hard to reverse-engineer

So, UML acts like a map of the system.

4. How does UML support scalability?

Yes, proper modelling makes systems easier to scale.

UML helps by:

- Showing modular structure (so new features can be added cleanly)
- Identifying weak coupling between components
- Supporting distributed system design
- Helping plan future expansion early

A well-modelled system is like a building with strong foundations—you can add more floors without collapsing it.

3. What is UML does not exist

1. What would happen if software projects were developed without UML or any modelling approach?

Possible consequences:

- Communication breakdown
Teams interpret requirements differently

- Requirement confusion
Developers build incorrect features
- Messy system structure
Poor architecture and tightly coupled code
- Integration failure
Modules don't work together properly
- Higher cost and delays
More bugs and redesign cycles

Overall, software development would become less predictable and more chaotic.

2. Which projects can survive without UML? Why?

Small apps: Yes, because they are simpler and easier to manage.

Start MVPs: Sometimes yes. Startups often focus on speed and testing ideas quickly.

Enterprise banking system : No. Banking systems are very large and require strong planning and security.

Hospital system : No. Hospital systems are important and sensitive, so mistakes can cause serious problems.

4. Reflection

Yes, I think UML is still important today.

Even though Agile development focuses on speed and AI tools can generate code quickly, developers still need good planning and communication.

UML helps teams understand the system before development starts. It is especially useful for large or complex projects.

However, modern teams may use simpler diagrams than before instead of creating too much documentation.

Overall, UML is still useful because good software design is important in every software project.